

Cursul 3

Client

IP_{server} : PORT_{server}

Server

metstat → aflăm
se vedem
porturile
libere

! trebuie verificat dacă s-a făcut:

- connect

- bind

SS - pe Linux mai
ușor

Local Area Network

ipconfig - Windows

ifconfig - Linux

ip addr

LAN

adrese IP private (false) (non-routabile)

WAN
INTERNET

Router

IP public
(real / routabil)

INTERNET

192.168.1.X
192.168.0.X
172.30.X.Y
diverse clase de adrese

! Obs: între 2 sau mai multe device-uri din
LAN nu este necesar routerul

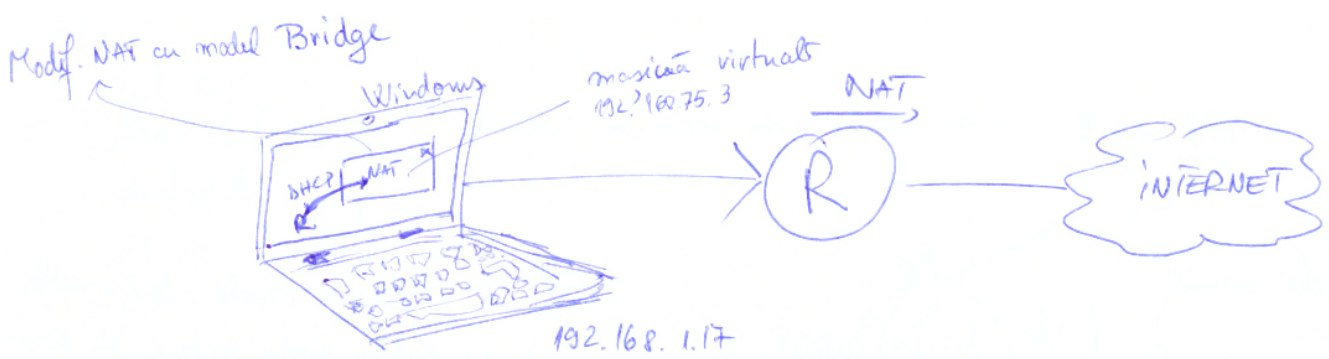
NAT
SNATNetwork Address Translation
Source — " —

→ dacă merge ping între două IP-uri, va funcționa conexiunea

IP C: 192.168.1.17

IP S: 192.168.1.37

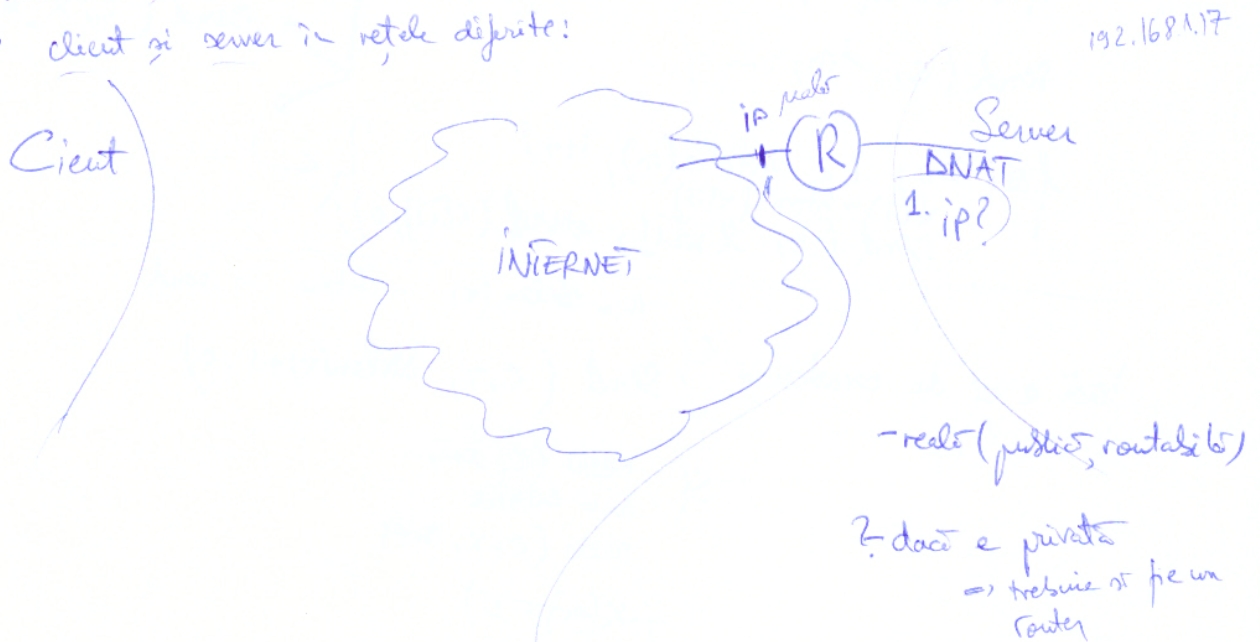
— nu e necesar să fie în aceeași rețea, doar observăm
că fac parte din aceeași clasă de adrese IP.



- Mașina fizică „jocă rolul de router” pentru mașina virtuală
- Dacă avem model Bridge pt. mașina virtuală (nu NAT, ca mai sus), mașina fizică nu mai e router pt. aceasta, căci însoțind Routerul (R) și va furniza adresa IP. (forma 192.168.1.X)
- Nu este obligatoriu ca serverul DHCP să ruleze pe router.

(când un device vrea să se conecteze la un router/server, trimite un mesaj de tip broadcast)

→ client și server în rețele diferite!

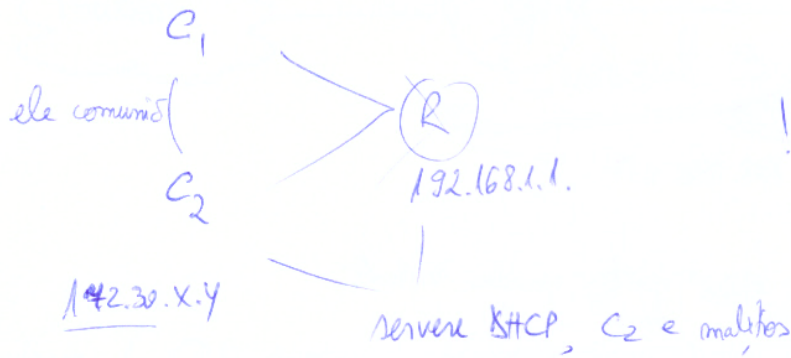


? dacă e privată
⇒ trebuie să fie un router

dacă cineva dorește să se conecteze la adresa IP a serverului și să fie redirecționat la adresa privată a serverului trebuie să specificăm adresa IP reală a routerului (extern) → clientul

DNAT
Port forwarding
Virtual Server

172.30.4.5 → nu va avea acces la internet !



! Obs : Adresele IP ale device-urilor dintr-o rețea trebuie să seamene cu cea a router-ului.

C → S

↓

uint16_t m;

n = htons(m)

send(c, &n, ...)

uint16_t *x;
x = malloc(sizeof(uint16_t))

send(c, x, sizeof(uint16_t), 0)

for (i = 0; i < ntohs(n); i++)
x[i] = htons(x[i]);
send(c, &x[i], sizeof(x[i]), 0);

n = strlen(x) → htons → send

bacă e sir de caractere : C) send(c, x, strlen(x)+1, 0)

S{
recv(c, &n, ...)
n = ntohs
recv(c, x, n+1
(x[n] = 0)

! Important : câte date trimitem, atâtea citim + de același tip de date (important pt. size)

ex:

```
int c;  
char s[100];  
uint16_t i, j;  
int x;  
c = socket ; connect --
```

```
scanf("%hu", &i)  
↓  
se citesc 2 octeti  
(hu - short unsigned)
```

```
// scanf("%d", &i)
```

! → pune la adresa lui i 4 octeti,
înseamnă i are doar 2 octeti
⇒ se scriu 4 octeti care pot
afecta alte date / variabile

strace / ltrace

./strace ./client